

Network Traffic Classification

A Comprehensive Survey of Techniques

From port-based classification to in-network deep learning

Introduction

Network traffic classification is the task of assigning a packet, flow, or session to a category — an application (Zoom, YouTube), a traffic type (voice, video, gaming, bulk), or a behavioral class (benign vs. malicious). It underpins QoS enforcement, billing, intrusion detection, traffic engineering, and capacity planning. The field has evolved through several generations of techniques as the network itself has changed: dynamic ports broke port-based classification, encryption (HTTPS, TLS 1.3, QUIC) broke payload inspection, and the sheer scale of modern traffic forced classification down into the data plane.

This report catalogues the full landscape of classification techniques in use today, organized by approach. For each technique we cover the underlying principle, representative production or research systems, strengths, limitations, and links to authoritative sources.

Taxonomy at a glance

Family	Examples	Encrypted?
Port-based	IANA registered ports, ACLs	No (broken)
Deep Packet Inspection (DPI)	NBAR2, App-ID, nDPI, Suricata	Partial
Protocol decoding	HTTP Host:, TLS SNI parsing	Partial
Statistical / behavioral	Packet size, IAT, rate, burst analysis	Yes
Flow-level ML	Random Forest, XGBoost, KNN	Yes
First-N packet methods	Bernaille (2006), early classification	Yes
TLS / QUIC fingerprinting	JA3, JA4, JA4+, JA4T	Yes (handshake)
Deep learning	CNN, LSTM, Deep Packet, ET-BERT	Yes
Knowledge distillation	DistilBERT, Mousika, ETKD, NetKD	Yes
In-network ML inference	Ilsy, Leo, Planter, Synecdoche	Yes
Graph-based	BLINC, GraphDApp, GNN approaches	Yes
Contextual / IBN	Cisco Catalyst Center, identity+device+app	Yes

1. Port-Based Classification

The simplest and oldest technique. Each application is assumed to run on a well-known TCP/UDP port: HTTP on 80, SSH on 22, SIP on 5060, DNS on 53. The router examines the 5-tuple (source IP, dest IP, source port, dest port, protocol) and looks up a table.

Why it fails today: Most modern applications multiplex over HTTPS (port 443), use dynamic ports, or deliberately obfuscate their port assignment (P2P, malware). The widespread adoption of HTTP/HTTPS as a universal transport mechanism for diverse application types — often multiplexed within a single connection — has undermined the reliability of port-based identification.

Where it still works: Tightly controlled environments (data centers with strict port hygiene), and as a fast-path first filter that defers ambiguous traffic to deeper analysis. Almost every modern system still uses port matching as the cheap initial check.

2. Deep Packet Inspection (DPI)

DPI examines packet payload bytes for known signatures or patterns. It works at OSI Layers 4 through 7, going beyond just headers. The classic signature for HTTP is the ASCII string 'GET ', 'POST ', or 'HEAD ' at a predictable offset; BitTorrent has the '\x13BitTorrent protocol' handshake.

2.1 Cisco NBAR / NBAR2

Network Based Application Recognition is Cisco's flagship classification engine, embedded in IOS since 2000. NBAR uses Protocol Description Language Modules (PDLs) — downloadable rule files — that describe how to recognize each application. NBAR2 is the next-generation rewrite, supporting 1500+ applications with less than 1% unknown traffic. NBAR2 uses Protocol Packs distributed outside the IOS release train for rapid signature updates, and integrates with SD-AVC cloud services for first-packet classification. Combines port matching, payload inspection, and behavioral heuristics.

Limitations: Cannot examine encrypted traffic such as HTTPS or IPsec. Cannot inspect later fragments of a packet. Examines only the first ~400 bytes of payload. Requires Cisco Express Forwarding.

2.2 Palo Alto App-ID

Palo Alto Networks' App-ID is a patented classification system that determines what an application is irrespective of port, protocol, or encryption. App-ID applies up to four mechanisms in sequence: (1) port + 5-tuple, (2) application signatures, (3) protocol decoding (e.g., parsing HTTP/SSH internals), and (4) heuristics for evasive applications. App-ID continually monitors application state midstream, supports parent/child app relationships (e.g., facebook-video as a child of facebook-base), and handles unknown traffic as its own explicitly managed category.

2.3 Juniper AppID / AppTrack

Juniper SRX firewalls implement AppID with a licensed signature database. Once enabled, AppID feeds AppTrack (visibility), AppFW (firewall rules), AppQoS (QoS prioritization), and APBR (advanced policy-based routing). The Application System Cache (ASC) accelerates repeated lookups.

2.4 Open-source DPI: nDPI, Suricata, Zeek

nDPI (used inside ntopng) is the de facto open-source DPI library — supports 250+ protocols with continuous community updates. **Suricata** is an open-source IDS/IPS that includes DPI as part of its

rule-based detection. **Zeek (formerly Bro)** takes a higher-level approach, parsing protocol semantics into structured event logs. **libprotoident** is a lightweight DPI library that uses the first 4 bytes of payload in each direction to identify protocols efficiently.

2.5 The encryption problem

DPI is increasingly blind. With TLS 1.3, QUIC (HTTP/3), and the upcoming Encrypted Client Hello (ECH), even handshake metadata is becoming opaque. ECH encrypts the SNI field of the Client Hello, removing the last reliable plaintext indicator of the destination service.

3. Protocol Decoding

More sophisticated than naive signature matching. Once a packet is identified as a protocol (say, HTTP), the classifier parses fields within that protocol to extract application context. Examples:

HTTP Host header: 'Host: youtube.com' identifies YouTube even on a non-standard port. Works for plaintext HTTP and any HTTPS proxy traffic that has been decrypted by the inspecting device.

TLS Server Name Indication (SNI): Sent in the Client Hello as plaintext, indicates which virtual host the client wants to reach. Until ECH deployment, SNI was the single most useful field for classifying HTTPS traffic.

QUIC initial handshake: The QUIC initial packet contains the SNI in its CRYPTO frame; tools like nDPI and Cisco SD-AVC parse this to identify QUIC-based services.

DNS query inspection: Often used as a side channel — observing 'A?youtube.com' queries from a host strongly predicts that subsequent HTTPS connections from that host are YouTube.

4. Statistical and Behavioral Classification

These methods compute statistics from packet/flow metadata — without ever reading payload — and use those statistics to identify applications. They work on encrypted traffic precisely because they don't look inside packets.

4.1 Per-packet features

Packet size: A foundational feature. VoIP codecs produce fixed-size packets (G.711 = 200 bytes, Opus = variable but small); video streams approach the MTU (1500 bytes); ACK-only TCP packets are 60 bytes. Internet traffic is bimodal — "mice" (small) and "elephants" (near-MTU). Packet size distribution is one of the most discriminating features in flow-based classification literature.

Direction (upstream/downstream): Web browsing is asymmetric (small request, large response); VoIP and gaming are symmetric.

TCP flag patterns: The sequence and frequency of SYN, ACK, FIN, PSH, RST flags is characteristic of different application behaviors.

4.2 Inter-packet times (IAT)

Inter-arrival time — the temporal gap between consecutive packets of a flow — is one of the most powerful encrypted-traffic features. Different applications produce dramatically different IAT signatures:

VoIP: Tight, regular IAT — typically 20ms (50 packets/sec) due to codec frame size. The IAT distribution looks like a near-flat line.

Video streaming: Large bursts of low-IAT packets during buffer fill, followed by long quiet periods — bimodal IAT distribution.

Gaming: Low, consistent IAT (often 30–60ms) with jitter spikes during action events.

Web browsing: Highly variable IAT — bursts during page load, long idle intervals while user reads.

Bulk file transfer: Steady stream at the connection's congestion-window-limited rate.

Statistical features extracted from IAT sequences include: mean, median, standard deviation, skew, min/max, and percentiles (p10, p25, p75, p90). The Moore dataset, a commonly used research benchmark, uses 249 such statistical features.

Caveat: Some research has shown that including IAT as an online feature can actually decrease classification accuracy for certain workloads (Impact of IAT Features for Online P2P Classification, 2018), because IAT measurement is sensitive to network jitter and timestamping precision. Hardware-assisted timestamping is preferred over software methods, which suffer OS scheduling delays.

4.3 Packet rate analysis

Closely related to IAT but expressed in the frequency domain: how many packets per unit time. Useful as a coarse, low-state classifier:

Constant bit rate (CBR): VoIP, fixed-rate video — packet rate is essentially constant. Detected by low variance in packet rate over a sliding window.

Variable bit rate (VBR): Adaptive streaming (DASH, HLS) — packet rate adjusts to available bandwidth. Detectable as periodic step changes corresponding to ABR algorithm decisions.

Bursty / bulk: File downloads, backups — packet rate equals link capacity divided by congestion-control state.

Sparse / interactive: SSH, telnet, gaming control packets — sparse stream punctuated by user input.

4.4 Burst analysis

A burst is defined as a sequence of consecutive packets whose inter-arrival times are below a threshold (commonly 10ms to 2s). Burst-based features are particularly powerful for mobile traffic where everything below the transport layer is encrypted by the cellular network. Researchers extract: (i) burst sizes, (ii) burst durations, (iii) inter-burst durations, (iv) instantaneous downlink throughput within bursts, and (v) packet count within bursts. Random Forest classifiers using these features achieve ~95% accuracy on real cellular networks; voice and video calling are easiest to identify, audio streaming hardest due to long inter-burst periods.

4.5 Flow-level features (5-tuple aggregation)

A flow is the unit of aggregation: packets sharing the same source IP, destination IP, source port, destination port, and transport protocol. Flow-level features summarize an entire flow:

Total packets, total bytes, flow duration, idle time, average packet size, average IAT, down-up byte ratio, segment size statistics, flow inter-arrival statistics.

Mice vs. elephant distinction: mice are short-lived, low-byte flows (20% of connections carry 80% of traffic — Zipf's law). Distinguishing mice from elephants drives traffic engineering.

4.6 First-N packet methods (early classification)

Bernaille et al. (2006) showed that an application can be identified by examining just the sizes of the first 5 data packets of each TCP flow. This enables very early classification — before the application has even revealed its full behavior. The technique is sensitive to packet loss and out-of-order delivery, but remains the basis of many production systems. Modern variants use the first 10–20 packets and add IAT and direction features.

5. TLS / QUIC Fingerprinting

Even when payload is encrypted, the TLS handshake itself is plaintext. Different TLS implementations (browsers, libraries, malware) advertise different cipher suites in different orders, making the Client Hello a fingerprint of the client software.

5.1 JA3 / JA3S

Introduced by Salesforce engineers in 2017. JA3 hashes five fields from the TLS Client Hello: TLS version, accepted cipher suites, list of extensions, elliptic curves, and EC point formats. Concatenated and MD5-hashed into a 32-character fingerprint. JA3S is the server-side counterpart (from Server Hello). Combined JA3+JA3S identifies a specific client/server software pair. Famously, Tor, Trickbot, Emotet, and many other malware families have stable, recognizable JA3 fingerprints.

Why JA3 is dying: Chrome, Firefox, and modern browsers now randomize the order of TLS extensions on every connection (TLS Extension Permutation). A single Chrome browser can produce billions of distinct JA3 hashes, breaking detection. Stamus Networks and Fastly have documented this in detail.

5.2 JA4 / JA4+ family

Developed by FoxIO (John Althouse, the JA3 creator) to solve the randomization problem. JA4 sorts ciphers and extensions before hashing, neutralizing permutation evasion. The fingerprint has structure: `t13d1516h2_8daaf6152771_e5627906d626` — Segment A (metadata: protocol, TLS version, SNI presence, counts), Segment B (sorted cipher hash), Segment C (sorted extensions hash). The JA4+ family includes JA4H (HTTP), JA4T (TCP), JA4L (light-distance / RTT), and variants for SSH and QUIC. Modern WAFs (Cloudflare, Akamai, DataDome) correlate JA4/JA4T/JA4H to detect inconsistencies (e.g., "User-Agent claims iOS but TCP fingerprint says Linux").

5.3 Future challenge: Encrypted Client Hello (ECH)

ECH (RFC 9180-based) encrypts the sensitive parts of the Client Hello, including SNI, splitting it into an outer ClientHello (plaintext, generic) and an inner ClientHello (encrypted, contains real destination). Once ECH is widely deployed (Cloudflare and Mozilla are pushing it), TLS handshake fingerprinting will lose much of its visibility into *where* traffic is going, though it will still see *which* client stack is making the request.

6. Classical Machine Learning

Once you have flow-level statistical features, train a supervised classifier. The standard ML algorithms applied to traffic classification:

Algorithm	Strengths	Where used
Decision Tree (C4.5, C5.0)	Interpretable, fast, maps to MAT	Ilsy, Mousika, Leo, Planter
Random Forest	High accuracy, handles imbalance	NetScraper, mobile burst classification
XGBoost / GBDT	Top accuracy on tabular features	Ilsy ensembles, FlowLens
SVM	Good with high-dim features	Moore-Zuev work, hybrid systems
Naive Bayes	Fast, low memory	Moore & Zuev (2005), Bayes net classifiers
KNN	Simple, no training	NetScraper, lightweight scenarios
K-means (unsupervised)	Cluster discovery, labeling	Bernaille first-5-packet clustering

Neural Network (shallow)	Universal approximator	NetScrapper ANN (99.86% on 53 apps)
--------------------------	------------------------	-------------------------------------

Feature engineering is critical. Williams et al. compared CFS, wrapper, and consistency feature selection on the NLANR dataset; Jenefa & Moses showed C5.0 with packet rate, data rate, and IAT statistics achieves 99% recall and precision across 17 applications.

7. Deep Learning Approaches

7.1 CNN-based — Deep Packet, 1D-CNN

Lotfollahi et al.'s **Deep Packet** (2020) treats the first N bytes of a packet as a 1D image and applies a 1D-CNN. Combines packet-level CNN with stacked autoencoders for both application identification and traffic characterization (browsing vs. streaming vs. P2P). FedETC uses 1D-CNN as the base model in a federated learning framework, avoiding manual feature engineering and preserving local data privacy.

7.2 RNN / LSTM — sequence modeling

Long Short-Term Memory networks model the temporal sequence of packet sizes and IATs. RIDS (INFOCOM 2024) uses an RNN co-designed with programmable switches. Useful for applications with long-range temporal dependencies (e.g., multi-stage attack flows).

7.3 Transformer / BERT-based — ET-BERT, NetConv, PERT

ET-BERT (Lin et al., WWW 2022) pre-trains a BERT-style transformer on large unlabeled traffic data, then fine-tunes for classification. Achieves state-of-the-art on five encrypted traffic tasks. Treats packet bytes as 'tokens' and learns a contextualized representation. **NetConv** adds byte-sequence pattern capture via stacked traffic convolution layers. **PERT** applies ALBERT (a lighter BERT variant) to encrypted traffic. **EDFormer** is a more recent transformer for Internet of Vehicles. The headline result: removing pre-training causes a 37.57% F1 drop, showing the value of self-supervised pre-training on unlabeled traffic.

7.4 Graph Neural Networks

Treat the network as a graph (hosts as nodes, flows as edges) and apply GNNs. **BLINC** (SIGCOMM 2005) was the original behavioral graph approach — visualizes connections between endpoints to distinguish client-server from P2P patterns. **GraphDApp** (IEEE TIFS 2021) uses GNNs to identify decentralized applications via encrypted traffic graphs. Recent work uses Curvature-aware GCN combined with attention-based capsule networks for multi-task traffic anomaly detection.

8. Knowledge Distillation for Traffic Classification

Knowledge distillation (Hinton et al., 2015) trains a compact 'student' model to mimic the output distribution of a complex 'teacher' model. The student achieves near-teacher accuracy with a fraction of the resources. This technique has become essential for deploying ML traffic classifiers in resource-constrained environments — programmable switches, embedded devices, mobile edge.

8.1 Why distillation matters here

A BERT-based encrypted traffic classifier may have 110M parameters and require GPU inference — obviously cannot run on a Tofino switch's match-action pipeline. Distilling the BERT teacher into a small decision tree or shallow MLP student lets you deploy the learned knowledge in hardware while preserving most of the accuracy.

8.2 General-purpose distilled models — DistilBERT, TinyBERT, MobileBERT

DistilBERT (Sanh et al., 2019) compresses BERT by 40% with 60% faster inference, retaining 97% of BERT's GLUE performance. Uses a triple loss: language modeling loss, distillation loss (matching teacher logits), and cosine-distance loss (matching teacher hidden states). TinyBERT and MobileBERT push the compression further. While not traffic-specific, these architectures are routinely used as the student

backbone in traffic classification distillation.

8.3 Traffic-specific distillation systems

Mousika (INFOCOM): distills complex ML models into a Binary Decision Tree (BDT) whose rules are encoded as ternary-match table entries on a P4 switch. Mousika additionally embeds the inferred Service Level Objective (SLO) into the packet header, enabling QoS-aware admission-controlled priority queuing. Most flows are correctly classified within the first 4 packets.

CyberSentinel (ArXiv 2024): distills an ensemble of autoencoders into an iForest (isolation forest), then converts iForest leaves into whitelist rules deployable on a Tofino data plane. First system to perform anomaly detection at line rate entirely in the data plane.

ETKD (Encrypted Traffic Knowledge Distillation): framework specifically designed to handle the accuracy-vs-complexity trade-off, achieving notably higher F1 than non-distilled SOTA models.

NetKD: unsupervised, task-agnostic, transformer-based knowledge distillation specifically for encrypted traffic. Uses Multi-Head Self-Attention Relation Alignment, Masked BURST Model, and Same-origin BURST Prediction tasks to transfer byte-level contextual knowledge into a compact NetKD-BERT student.

Edge-Mamba (2026): linear-time selective State Space architecture distilled for IoT intrusion detection — runs on end devices in real time across heterogeneous datasets.

Mousika v2: introduces a teacher-student knowledge distillation architecture that enables transfer from sophisticated models to BDT — sophisticated models are 'indirectly deployed' in switches. Improves classification accuracy by 3.27% while reducing switch stage and memory usage by 2x.

9. In-Network ML Inference (Data-Plane Classification)

The newest frontier: actually running the classifier on the switch ASIC at line rate. Programmable data planes (P4 / PISA architecture) have enabled this. The standard recipe: train a tree-based classifier offline, compile the tree to match-action table entries, deploy to the switch.

9.1 Foundational systems

Illy (HotNets 2019, Oxford 2024): the canonical decision-tree-to-MAT mapping. Uses one table per feature, with each table encoding which branch of the tree was taken; a final table combines the branch codes into a class label. Supports DT, RF, XGBoost, SVM, Naive Bayes, K-means.

Planter: generalizes to ensemble methods (Random Forest, XGBoost) and provides a Python toolchain to compile arbitrary scikit-learn models to P4.

Leo (NSDI 2023): online traffic classification at multi-terabit line rates on Intel Tofino. Crucially supports runtime model updates without switch downtime — the model can be retrained daily and reloaded on-the-fly.

FlowLens (NDSS 2021): efficient flow distribution capture on Tofino. Computes a memory-efficient 'flow marker' summarizing the per-flow distribution of features, then classifies using ML. Single P4 program supports multiple downstream classification tasks at runtime without redeployment.

SwitchTree, pForest: alternative decision-tree and random-forest-on-P4 implementations.

9.2 Recent advances (2024–2026)

Synecdoche (ArXiv 2026): direct packet sequential pattern matching on Tofino. Achieves F1-score improvements of up to 26.4% over statistical methods and 18.3% over online deep learning approaches, with 79.2% reduction in SRAM usage and 13% reduction in latency.

Brain-on-Switch (NSDI 2024): NN-driven traffic analysis at line speed in the data plane.

Jewel (INFOCOM 2024): joint packet-level and flow-level inference in a single programmable switch.

Henna: in-switch hierarchical classification system aligned with the PISA architecture.

AdaFlow: incorporates traffic-specific heuristics into the in-network classifier design.

AI-INT (Springer 2025): combines P4 with ONOS controller and a Multi-Layer Perceptron running on a BMv2 switch. Predicts traffic class on flow detection; controller installs telemetry and forwarding rules accordingly.

10. Contextual / Intent-Based Classification

Modern enterprise networks classify by combining traffic features with non-traffic context: user identity, device type, time of day, location, compliance posture. This is the approach taken by **Cisco Catalyst Center (formerly DNA Center)** and Cisco SD-Access:

Cisco ISE authenticates the user/device (802.1X, MAC Authentication Bypass, profiling) and assigns Security Group Tags (SGTs).

NBAR2 identifies the application via DPI/flow signatures.

DNA Center combines identity + application + time-of-day into a unified policy decision.

Switches apply SGT-based ACLs and QoS — "contractor can access web frontend but not the database during the maintenance window."

The Cisco DNA Assurance engine adds AI/ML-based root cause analysis on the resulting telemetry. Comparable systems: Aruba ClearPass + Central, Juniper Mist with Marvis, Cisco Meraki Insight.

11. Emerging Techniques and Open Challenges

11.1 Federated learning for traffic classification

FedETC: multiple participants jointly train a global encrypted-traffic classifier without sharing raw traffic. Uses 1D-CNN as the base model. Achieves accuracy close to centralized training while preserving local privacy — important for ISPs and carriers reluctant to share customer traffic.

11.2 Active learning & concept drift

MicFoal: multiclass imbalanced + concept drift framework with online active learning. Uses a configurable supervised learner, hybrid label request strategy, label sliding window group, and adaptive label cost budget. Tackles the reality that traffic patterns change over time as applications evolve.

11.3 Self-supervised learning

SAFE introduces self-supervised flow representation learning that leverages trace-level (mixture of correlated flows) information. Reduces the need for manual labeling — important since labeled traffic datasets are scarce and expire quickly.

11.4 The QUIC / HTTP/3 challenge

QUIC multiplexes streams over UDP and encrypts most of its handshake. Traditional flow-based classification still works (packet sizes and IATs leak through encryption), but distinguishing QUIC-based video from QUIC-based browsing is harder than the TCP equivalent. JA4 has explicit QUIC support; this is an active research area.

11.5 The ECH end-game

Encrypted Client Hello will eventually remove SNI as a classification signal. The community will be forced fully into statistical / behavioral / fingerprinting techniques. Expect renewed focus on purely-timing-based classification, JA4T (TCP-layer fingerprinting), and ML approaches that don't depend on any plaintext metadata.

12. Vendor System Cheat-Sheet

Vendor / System	Primary technique	Encrypted handling
Cisco NBAR2 (IOS, IOS-XE)	PDLM signatures + protocol packs + SD-AVC cloud	Limited; uses TLS metadata
Cisco Catalyst Center	NBAR2 + ISE identity + contextual policy	Yes via context layer
Palo Alto App-ID	Signatures + decoders + heuristics, 2000+ apps	Yes via decryption + behavioral
Juniper AppID / AppTrack	Signature DB subscription + ASC cache	Limited
Fortinet FortiGuard	Application Control + SSL inspection	Yes via decryption
nDPI (open source)	Signature + heuristic library, 250+ protocols	TLS SNI parsing
Suricata	Rule-based DPI + ML extensions	Limited
Zeek (Bro)	Protocol parsing into structured event logs	Limited

Cisco Meraki	NBAR2 cloud-updated, 1500+ applications	TLS metadata + behavioral
Cloudflare WAF	JA3/JA4 + behavioral + rate analysis	Yes (fingerprinting)

References & Sources

Vendor Systems

Cisco NBAR product page

<https://www.cisco.com/c/en/us/products/ios-nx-os-software/network-based-application-recognition-nbar/index.html>

LiveAction NBAR / NBAR2 explainer

<https://www.liveaction.com/resources/blog-post/what-is-network-based-application-recognition-nbar/>

Meraki NBAR2 + SD-AVC documentation

[https://documentation.meraki.com/General_Administration/Cross-Platform_Content/Next-gen_Traffic_Analytics_-_Network-Based_Application_Recognition_\(NBAR\)_Integration](https://documentation.meraki.com/General_Administration/Cross-Platform_Content/Next-gen_Traffic_Analytics_-_Network-Based_Application_Recognition_(NBAR)_Integration)

NBAR community deep-dive (Cisco Community)

<https://community.cisco.com/t5/networking-knowledge-base/network-based-application-recognition-nbar/ta-p/3116062>

Network-Based Application Recognition (Wikipedia)

https://en.wikipedia.org/wiki/Network_Based_Application_Recognition

NBAR (CIO Wiki)

[https://cio-wiki.org/wiki/Network_Based_Application_Recognition_\(NBAR\)](https://cio-wiki.org/wiki/Network_Based_Application_Recognition_(NBAR))

Palo Alto App-ID Overview (PAN-OS docs)

<https://docs.paloaltonetworks.com/ngfw/administration/app-id/app-id-overview>

Palo Alto App-ID Tech Brief

https://media.paloaltonetworks.com/documents/App_ID_tech.pdf

Palo Alto App-ID, User-ID, ACC Features (Firewall.cx)

<https://www.firewall.cx/security/palo-alto-networks/palo-alto-firewall-app-id-user-id-application-control-centre.html>

Palo Alto App-ID How It Works (PacketSwitch)

<https://www.packetswitch.co.uk/palo-alto-app-id-configuration/>

Juniper Application Identification on SRX

<https://www.juniper.net/documentation/us/en/software/junos/application-identification/topics/topic-map/security-application-identification-overview.html>

Juniper AppID for Unified Policies

<https://www.juniper.net/documentation/us/en/software/junos/application-identification/topics/topic-map/application-identification-unified-policy.html>

Cisco Catalyst Center / DNA Center IBN Results

<https://www.cisco.com/c/en/us/solutions/cisco-on-cisco/cisco-dna-center-results.html>

Cisco SD-Access Segmentation Design Guide

<https://community.cisco.com/t5/networking-knowledge-base/sd-access-segmentation-design-guide/ta-p/4935734>

Cisco IBN and IoT White Paper

<https://www.cisco.com/c/en/us/solutions/collateral/internet-of-things/nb-09-intent-based-iot-wp-cte-en.html>

Statistical / Flow-Based Classification

Tutorial on Flow-Based Network Traffic Classification with ML (ArXiv 2026)

<https://arxiv.org/html/2601.04089v1>

NetScraper: Real-Time ML Traffic Classifier (MDPI 2021)

<https://www.mdpi.com/1999-4893/14/8/250>

NetScraper NSF Public Access record

<https://par.nsf.gov/biblio/10330675-real-time-network-traffic-classifier-online-applications-using-machine-learning>

High Performance Traffic Classification: Message Size Sequence (ScienceDirect)

<https://www.sciencedirect.com/science/article/abs/pii/S108480451630220X>

Flow Classification Overview (ScienceDirect Topics)

<https://www.sciencedirect.com/topics/computer-science/flow-classification>

Traffic Classification Overview (ScienceDirect Topics)

<https://www.sciencedirect.com/topics/computer-science/traffic-classification>

Classifying Traffic Overview (ScienceDirect Topics)

<https://www.sciencedirect.com/topics/computer-science/classifying-traffic>

Mobile Traffic Classification through Burst Statistical Features

<https://imt-atlantique.hal.science/hal-04154479/document>

Impact of Packet Inter-arrival Time Features for Online P2P Classification

<https://www.researchgate.net/publication/326403507>

Fast Traffic Classification using Joint Distribution of Packet Size + EPPT (IEICE)

https://globals.ieice.org/en_transactions/information/10.1587/transinf.E93.D.2944/_p

Traffic classification: packet-, flow-, and application-based survey

<https://scispace.com/pdf/traffic-classification-packet-flow-and-application-based-3sc6aa3yih.pdf>

SDN-based Traffic Classification using ML (Nature Sci. Reports 2024)

<https://www.nature.com/articles/s41598-024-70983-6>

SDN Traffic Classification for QoS Optimization with ML (Springer 2025)

<https://link.springer.com/article/10.1007/s10922-025-09911-6>

TLS / QUIC Fingerprinting

TLS Fingerprinting with JA3 and JA3S (Salesforce Engineering)

<https://engineering.salesforce.com/tls-fingerprinting-with-ja3-and-ja3s-247362855967/>

JA3/JA4 TLS Fingerprint detector (Scrapfly)

<https://scrapfly.io/web-scraping-tools/ja3-fingerprint>

TLS Fingerprinting JA3/JA4 detection guide (Wilico)

<https://wilico.co.jp/en/blog/tls-fingerprint-ja3-ja4-detection>

JA4 Fingerprinting: From Black Boxes to Beacons (Hunt.io)

<https://hunt.io/glossary/ja4-fingerprinting>

Cloudflare JA3/JA4 fingerprint documentation

<https://developers.cloudflare.com/bots/additional-configurations/ja3-ja4-fingerprint/>

All you need to know about JA3 / JA4 (Medium)

<https://medium.com/@ggabrielhd/all-you-need-to-know-about-ja3-ja4-fingerprints-and-how-to-collect-them-8f189085b61f>

TLS Fingerprinting in 2026: JA3, JA4+, and ECH (packet.guru)

<https://packet.guru/blog/TLS-Fingerprinting-JA3-JA4>

Adapting to Change: JA3 Fingerprints Fade (Stamus Networks)

<https://www.stamus-networks.com/blog/ja3-fingerprints-fade-browsers-embrace-tls-extension-randomization>

FoxIO JA4+ project (GitHub)

<https://github.com/FoxIO-LLC/ja4>

JA3 (Salesforce GitHub repo)

<https://github.com/salesforce/ja3>

TLS Fingerprinting How-to-Bypass Guide (Browserless 2025)

<https://www.browserless.io/blog/tls-fingerprinting-explanation-detection-and-bypassing-it-in-playwright-and-puppeteer>

Deep Learning for Encrypted Traffic

ET-BERT: Contextualized Datagram Representation (WWW 2022)

<https://dl.acm.org/doi/fullHtml/10.1145/3485447.3512217>

ET-BERT (Semantic Scholar entry with ETKD reference)

<https://www.semanticscholar.org/paper/ET-BERT:-A-Contextualized-Datagram-Representation-Lin-Xiong/29ed68d701f8450853938827b5124c9613c56aff>

Encrypted Network Traffic Analysis ML Survey (PMC 2024)

<https://pmc.ncbi.nlm.nih.gov/articles/PMC11175201/>

ET-BERT (ResearchGate)

https://www.researchgate.net/publication/360178822_ET-BERT_A_Contextualized_Datagram_Representation_with_Pre-training_Transformers_for_Encrypted_Traffic_Classification

Knowledge Distillation

DistilBERT, A Distilled Version of BERT (Hugging Face)

https://huggingface.co/docs/transformers/model_doc/distilbert

Knowledge Distillation in BERT (Medium)

<https://medium.com/@avinashmachinelearninginfo/knowledge-distillation-in-bert-527cd243e9e2>

Mousika: General In-Network Intelligence by Knowledge Distillation

https://www.researchgate.net/publication/361444527_Mousika_Enable_General_In-Network_Intelligence_in_Programmable_Switches_by_Knowledge_Distillation

Mousika (smart Internet group PDF)

<https://smartinternet.group/wp-content/uploads/2024/01/paper-xgr-mousika-infocom.pdf>

CyberSentinel: Knowledge-Distilled Anomaly Detection on P4 (ArXiv 2024)

<https://arxiv.org/abs/2412.16693>

CyberSentinel ResearchGate version

https://www.researchgate.net/publication/387350983_CyberSentinel_Efficient_Anomaly_Detection_in_Programmable_Switch_using_Knowledge_Distillation

In-Network Machine Learning

IlSy: Practical In-Network Classification (ArXiv 2022)

<https://arxiv.org/pdf/2205.08243>

IlSy expanded version (Oxford 2024)

<https://eng.ox.ac.uk/media/km0etg1w/zheng24iisy.pdf>

IlSy original HotNets paper PDF

<https://eng.ox.ac.uk/media/11760/zheng22iisy.pdf>

IlSy GitHub

<https://github.com/cucl-srg/IlSy>

IlSy Semantic Scholar

<https://www.semanticscholar.org/paper/IlSy:-Practical-In-Network-Classification-Zheng-Xiong/02b0afdf8ecd63b4942f024e812903797be13fa5>

Synecdoche: Direct Sequential Pattern Matching (ArXiv 2026)

<https://arxiv.org/html/2512.21116v2>

FlowLens: Efficient Flow Classification on Tofino (NDSS 2021)

https://www.ndss-symposium.org/wp-content/uploads/ndss2021_7C-2_24067_paper.pdf

Real-Time Encrypted Traffic Classification with P4 + ML (IJNM)

<https://dl.acm.org/doi/abs/10.1002/nem.2320>

A Framework for In-Network Inference using P4 (ARES 2024)

https://deterministic6g.eu/images/publications/A_Framework_for_In-network_Inference_using_P4.pdf

Montimage MAIP — open-source DT-on-P4 working code

<https://github.com/Montimage/in-network-inference-using-p4>

In-Network ML Survey (Oxford 2023)

<https://eng.ox.ac.uk/media/vd5kor0j/zheng23survey.pdf>

AI-INT: AI-Driven Telemetry for Programmable Networks (Springer 2025)

https://link.springer.com/chapter/10.1007/978-3-032-16632-6_20

P4-NIDS: High-Performance Monitoring + IDS in P4 (ArXiv 2024)

<https://arxiv.org/html/2411.17987v1>

Review of P4 Programmable Data Planes for Network Security (Wiley 2021)

<https://onlinelibrary.wiley.com/doi/10.1155/2021/1257046>

Exhaustive Survey on P4 Switches (Kfoury et al.)

https://research.cec.sc.edu/files/cyberinfra/files/an_exhaustive_survey_on_p4_programmable_data_plane_switches_taxonomy_applications_challenges_and_future_tren.pdf

P4 ML Toolbox (ResearchGate)

https://www.researchgate.net/publication/380654457_A_Machine_Learning-Based_Toolbox_for_P4_Programmable_Data-Planes